3次ベジェ曲線の特異点の解析的導出*

場生松 武志†

2026年10月31日

概 要

3次ベジェ曲線は、制御点の配置によっては自己交差、カスプ（尖点）、変曲点などの特異点が生じる場合がある。ベジェ曲線の基本的な性質は十分に文書化され、実用的に広く利用されている [1] ものの、特異点の明示的な定式化は、従来の文献には必ずしも見当たらない。本稿では、これについて解析的な表現式を導出する。また、各特異点の判別と導出の統合テーブルを提示する。さらに、実用への適用を容易にするためC言語による実装例を提示する。

1 背景と関連研究

3次ベジェ曲線は滑らかな曲線をモデリングするために広く用いられているが、産業用途においては、自己交差やカスプの混入が重大な影響を及ぼすおそれがあり、入力データに対しこれらの非混入を保証することが求められる。変曲点についてもデザイン品質などの観点から非混入の保証を求められる場合がある。入力データにはしばしば膨大な数のベジェ曲線が含まれ、これらを迅速に処理するための軽量な方式が必要である。

3次ベジェ曲線の特異点の計算には、曲線のパラメトリック関数や曲率から導かれる多項式を解くことが必要になる。StoneとDeRoseは特異点を類型化し、ベジェ曲線を正準形式に変換することで各類型がXY平面上にマッピングされ、判別できることを示した [2] が、正準形式に変換するための計算量が多いことや、正準形式に起因する制約で$t = 0$付近の変曲点が扱えず制御点の並び順を反転する前処理が必要、などの欠点があった。白山は自己交差の交点を制御点の座標から直接求める解析的な表現式を示した [3]。本稿ではこれを元に演算誤差を低減する改良を行う。また、変曲点を求める表現式を導出し、各特異点の判別と導出の統合化をはかる。

2 準備

3次ベジェ曲線上の任意の点の座標を、

$$x = f(t) = a_0t^3 + a_1t^2 + a_2t + a_3$$
$$y = g(t) = b_0t^3 + b_1t^2 + b_2t + b_3$$

*本稿は、以下で公表済みの内容に基づいて再構成したものである。
Qiita「3次ベジェ曲線の特異点の解析的導出」
https://qiita.com/kumanomi1000/items/4ba97d0b24ce06f4fb1d

†bashomatsutakeshi@gmail.com

とする。ただし $t \in [0,1]$、a_i と b_i は、3次ベジェ曲線の定義 [4] により制御点 $\boldsymbol{B}_0, \boldsymbol{B}_1, \boldsymbol{B}_2, \boldsymbol{B}_3$ について

$$
\begin{aligned}
a_0 &= -B_{0x} + 3B_{1x} - 3B_{2x} + B_{3x} \\
a_1 &= 3B_{0x} - 6B_{1x} + 3B_{2x} \\
a_2 &= -3B_{0x} + 3B_{1x} \\
a_3 &= B_{0x} \\
b_0 &= -B_{0y} + 3B_{1y} - 3B_{2y} + B_{3y} \\
b_1 &= 3B_{0y} - 6B_{1y} + 3B_{2y} \\
b_2 &= -3B_{0y} + 3B_{1y} \\
b_3 &= B_{0y}
\end{aligned}
$$

とする。また、特異点を求めるための係数を

$$
\begin{aligned}
v &= a_0 b_1 - a_1 b_0 \\
w &= a_0 b_2 - a_2 b_0 \\
u &= a_1 b_2 - a_2 b_1
\end{aligned}
$$

とする。

3　自己交差の交点とカスプ点

自己交差が発生する条件は、$f(t_1) = f(t_2)$ と $g(t_1) = g(t_2)$ を同時に満たす、異なるパラメータ t_1 と t_2 が存在することである。x については

$$
\begin{aligned}
&a_0 t_1^3 + a_1 t_1^2 + a_2 t_1 + a_3 = a_0 t_2^3 + a_1 t_2^2 + a_2 t_2 + a_3 \\
&a_0 (t_1^3 - t_2^3) + a_1 (t_1^2 - t_2^2) + a_2 (t_1 - t_2) = 0
\end{aligned}
$$

両辺を $(t_1 - t_2)$ で割ると

$$
a_0 (t_1^2 + t_1 t_2 + t_2^2) + a_1 (t_1 + t_2) + a_2 = 0 \tag{1}
$$

y についても同様に

$$
b_0 (t_1^2 + t_1 t_2 + t_2^2) + b_1 (t_1 + t_2) + b_2 = 0 \tag{2}
$$

(1) を変形して (2) に代入し、t_1 について整理する。

$$
\begin{aligned}
&\frac{b_0}{a_0}(-a_1(t_1 + t_2) - a_2) + b_1(t_1 + t_2) + b_2 = 0 \\
&t_1 = -t_2 - \frac{b_0 a_2 - b_2 a_0}{b_0 a_1 - b_1 a_0} \\
&t_1 = -t_2 - \frac{w}{v}
\end{aligned}
$$

(1) に代入し

$$
a_0 t_2^2 + \frac{a_0 w t_2}{v} + \frac{a_0 w^2}{v^2} - \frac{a_1 w}{v} + a_2 = 0
$$

両辺に v/a_0 を掛けると

$$vt_2^2 + wt_2 + \frac{w^2}{v} - \frac{a_1 w}{a_0} + \frac{a_2 v}{a_0} = 0 \tag{3}$$

を得る。$v = 0$ の場合、自己交差やカスプは存在しない。$v \neq 0$ の場合、2 次方程式の解の公式にあてはめ、t_1 と t_2 の対称関係を用いることで

$$t_1, t_2 = \frac{-w \pm \sqrt{-3w^2 + 4vu}}{2v}$$

を得る。$t_1 \in [0,1] \wedge t_2 \in [0,1]$ である場合、自己交差が存在する。$-3w^2 + 4vu = 0$ である場合は $t_1 = t_2$ となり、カスプになる。

4 変曲点

パラメトリック曲線の曲率は

$$\kappa = \frac{x'y'' - y'x''}{\left(x'^2 + y'^2\right)^{3/2}},$$

であり [5]、変曲点はその分子 $\kappa_a = x'y'' - y'x''$ がゼロになる点である。x と y の 1 次微分と 2 次微分

$$\begin{aligned}
x' &= 3a_0t^2 + 2a_1t + a_2 \\
x'' &= 6a_0t + 2a_1 \\
y' &= 3b_0t^2 + 2b_1t + b_2 \\
y'' &= 6b_0t + 2b_1
\end{aligned}$$

を κ_a に代入すると、

$$\begin{aligned}
&\kappa_a = 3(a_1b_0 - a_0b_1)t^2 + 3(a_2b_0 - a_0b_2)t + a_2b_1 - a_1b_2 = 0 \\
&3vt^2 + 3wt + u = 0
\end{aligned} \tag{4}$$

2 次方程式の解の公式にあてはめ、

$$t = \frac{-3w \pm \sqrt{3(3w^2 - 4uv)}}{6v}$$

を得る。また、$v = 0 \wedge w \neq 0$ の場合、(4) は 1 次式になり、

$$t = -\frac{u}{w}$$

である。それぞれ $t \in [0,1]$ である場合、変曲点である。なお $v = 0 \wedge w = 0$ の場合、変曲点は存在しない。

5　判定の統合

上記各導出式には共通項 $D = 4uv - 3w^2$ が含まれる。これを判別式と捉えることで、以下の統合テーブルにまとめることができる。

$D < 0$	変曲点	$v \neq 0$	$t = \dfrac{-w \pm \sqrt{-\frac{D}{3}}}{2v}$
		$v = 0 \wedge w \neq 0$	$t = -\dfrac{u}{3w}$
$D = 0 \wedge v \neq 0$	カスプ		$t = -\dfrac{w}{2v}$
$D > 0 \wedge v \neq 0$	自己交差		$t = \dfrac{-w \pm \sqrt{D}}{2v}$

6　考察

高次多項式の計算では浮動小数演算に起因する計算誤差の影響が顕著になる可能性がある。本稿の各計算式では制御点の座標値については 4 乗以下としており、この影響をある程度抑えているが、制御点の座標の値域によっては影響が顕在化する可能性があり、座標値をある範囲内になるよう変換して特異点を導出するすることが望ましい。添付する実装例では、すべての座標値を 0 から 1 の範囲内に正規化する前処理を適用している。

7　結論

本稿では、3 次ベジェ曲線の自己交差点、カスプ点、変曲点の解析的な表現式と、判別と導出の統合テーブルを提示した。C 言語による実装例を Appendix に提示してある。

参考文献

[1] P. Bézier. *Mathematical and Practical Possibilities of UNISURF*. Computer Aided Geometric Design - Proceedings of a Conference Held at The University of Utah, Salt Lake City, Utah, March 18-21, 1974, pp.127-152. ISBN 0-12-079050-5.

[2] Maureen C. Stone, Tony D. DeRose. *A Geometric Characterization of Parametric Cubic Curves*. ACM Transactions on Graphics, Vol. 8, No. 3, July 1989, pp.147-163.

[3] Zenn 白山風露『自己交差する 3 次ベジェ曲線の交差点を求める』
https://zenn.dev/kazatsuyu/scraps/a362d9d1c35da6

[4] Gerald Farin. *Curves and Surfaces for Computer Aided Geometric Design: A Practical Guide*. Academic Press, ISBN 0-12-249050-9, p.42, 1988.

[5] Wikipedia – Curvature
https://en.wikipedia.org/wiki/Curvature

Appendix A　実装例ソースコード

ソースコード 1: bzsingular.cpp

```
#include <cmath>
#include <QPointF>
#include <QList>
using namespace std;

#define EZERO 1e-14 // これより 0に近い値は 0とする

#define PW3(x) ((x)*(x)*(x))
#define PW2(x) ((x)*(x))

// 3次ベジェ曲線の係数を求める
void coefBezier(
  QList<QPointF> &cp, // IN ベジェ曲線 制御点
  double *a, // OUT x 係数
  double *b) // OUT y 係数
{
   a[0]=-cp[0].x()+3*cp[1].x()-3*cp[2].x()+cp[3].x();
   a[1]=3*cp[0].x()-6*cp[1].x()+3*cp[2].x();
   a[2]=-3*cp[0].x()+3*cp[1].x();
   a[3]=cp[0].x();
   b[0]=-cp[0].y()+3*cp[1].y()-3*cp[2].y()+cp[3].y();
   b[1]=3*cp[0].y()-6*cp[1].y()+3*cp[2].y();
   b[2]=-3*cp[0].y()+3*cp[1].y();
   b[3]=cp[0].y();
}

// 3次パラメトリック曲線の座標を求める
QPointF pointBezier(
  double *a, // IN x 係数
  double *b, // IN y 係数
  double t) // IN パラメータ
{
   return QPointF(a[0]*PW3(t)+a[1]*PW2(t)+a[2]*t+a[3],
               b[0]*PW3(t)+b[1]*PW2(t)+b[2]*t+b[3]);
}

// 制御点の座標値を [0,1]に正規化
void normCP(
  QList<QPointF> &cp, // IN 曲線の制御点
  QList<QPointF> &ncp) // OUT 曲線の制御点(正規化)
{
   double xmin=cp[0].x();
   double xmax=cp[0].x();
   double ymin=cp[0].y();
   double ymax=cp[0].y();
   foreach(QPointF p,cp.mid(1)){
      if(xmin>p.x()) xmin=p.x();
      if(xmax<p.x()) xmax=p.x();
```

```
      if(ymin>p.y()) ymin=p.y();
      if(ymax<p.y()) ymax=p.y();
   }
   double m=max(xmax-xmin,ymax-ymin);
   ncp.clear();
   foreach(QPointF p,cp)
      ncp<<QPointF((p.x()-xmin)/m,(p.y()-ymin)/m);
}

// 3次ベジェ曲線の特異点を求める
bool vBezierSingular( // RETURN true:正常
  QList<QPointF> &cp, // IN 3次ベジェ曲線 制御点 (4点)
  QList<QPointF> &opI, // OUT 変曲点座標 (0〜2点)
  QList<QPointF> &opC, // OUT カスプ点座標 (0または1点)
  QList<QPointF> &opS) // OUT 自己交差点座標 (0または1点)
{
   opI.clear();
   opC.clear();
   opS.clear();
   if(cp.size()!=4)
      return false;

   // ベジェ曲線の係数を求める
   double a[4],b[4];
   coefBezier(cp,a,b);

   // 制御点の正規化座標と係数を求める
   QList<QPointF> ncp;
   normCP(cp, ncp);
   double na[4],nb[4];
   coefBezier(ncp,na,nb);

   // 特異点を求める
   double v=na[0]*nb[1]-na[1]*nb[0];
   double w=na[0]*nb[2]-na[2]*nb[0];
   double u=na[1]*nb[2]-na[2]*nb[1];
   double D=4*u*v-3*PW2(w);
   if(abs(v)>=EZERO){
      if(D<=-EZERO){
         // 変曲点
         double sqd=sqrt(-D/3);
         double t1=(-w+sqd)/2/v;
         double t2=(-w-sqd)/2/v;
         if(t1>=0&&t1<=1)
            opI<<pointBezier(a,b,t1);
         if(t2>=0&&t2<=1)
            opI<<pointBezier(a,b,t2);
      }else if(D<EZERO){
         // カスプ
         double t=-w/2/v;
         if(t>=0&&t<=1)
```

```
            opC<<pointBezier(a,b,t);
        }else{
            // 自己交差
            double sqd=sqrt(D);
            double t1=(-w+sqd)/2/v;
            double t2=(-w-sqd)/2/v;
            if(t1>=0&&t1<=1&&t2>=0&&t2<=1)
                opS<<pointBezier(a,b,t1);
        }
    }else{
        if(D<=-EZERO&&abs(w)>=EZERO){
            // 変曲点
            double t=-u/3/w;
            if(t>=0&&t<=1)
                opI<<pointBezier(a,b,t);
        }
    }
    return true;
}
```

Appendix B　Qt サンプルアプリ

起動するとカスプを含む 3 次ベジェ曲線と、カスプ点が水色で表示される。制御点をドラッグして移動すると曲線が更新され特異点が再検出される。自己交差があれば赤色、変曲点があれば緑色で表示される。

実行例：



ソースコード 2: mainwindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QGraphicsScene>
#include <QGraphicsItem>
#include <QGraphicsSceneMouseEvent>
```

```
QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
QT_END_NAMESPACE

class Scene : public QGraphicsScene
{
   Q_OBJECT
public:
   explicit Scene(QObject *parent = nullptr){ Q_UNUSED(parent) }

signals:
   void mouseMove(QGraphicsSceneMouseEvent *mouseEvent);
   void mousePress(QGraphicsSceneMouseEvent *mouseEvent);

protected:
   virtual void mouseMoveEvent(QGraphicsSceneMouseEvent *mouseEvent)
      override
   { emit mouseMove(mouseEvent); }
   virtual void mousePressEvent(QGraphicsSceneMouseEvent *mouseEvent)
      override
   { emit mousePress(mouseEvent); }
};

class MainWindow : public QMainWindow
{
   Q_OBJECT

public:
   MainWindow(QWidget *parent = nullptr);
   ~MainWindow();

private slots:
   void mouseMove(QGraphicsSceneMouseEvent *mouseEvent);
   void mousePress(QGraphicsSceneMouseEvent *mouseEvent);

private:
   Scene scene;
   QGraphicsEllipseItem *rc[4];
   QGraphicsPathItem *ci;
   QList<QGraphicsEllipseItem *> ri;
   QList<QPointF> cp;
   int scp;

private:
   Ui::MainWindow *ui;
};
#endif // MAINWINDOW_H
```

ソースコード 3: mainwindow.cpp

```
#include "mainwindow.h"
```

```
#include "ui_mainwindow.h"

bool vBezierSingular(
   QList<QPointF> &cp, // IN 3次ベジェ曲線 制御点 (4点)
   QList<QPointF> &opI, // OUT 変曲点座標 (0〜2点)
   QList<QPointF> &opC, // OUT カスプ点座標 (0または1点)
   QList<QPointF> &opS); // OUT 自己交差点座標 (0または1点)

MainWindow::MainWindow(QWidget *parent)
: QMainWindow(parent)
, ui(new Ui::MainWindow)
{
   ui->setupUi(this);
   scene.setSceneRect(0,0,ui->graphicsView->width()-2,ui->graphicsView->
      height()-2);
   ui->graphicsView->setScene(&scene);
   ui->graphicsView->setRenderHint(QPainter::Antialiasing);
   // example of cusp
   cp<<QPointF(120,50);
   cp<<QPointF(120,150);
   cp<<QPointF(220,150);
   cp<<QPointF(20,50);
   for(int i=0;i<4;i++){
      rc[i]=scene.addEllipse(cp[i].x()-2.5,cp[i].y()-2.5,5,5,QPen(Qt::blue
         ));
   }
   {
      QPainterPath curve(cp[0]);
      curve.cubicTo(cp[1],cp[2],cp[3]);
      ci=scene.addPath(curve,QPen(QBrush(Qt::blue),1));
   }
   QList<QPointF> opI,opC,opS;
   vBezierSingular(cp,opI,opC,opS);
   foreach (QPointF p,opI)
      ri<<scene.addEllipse(p.x()-2.5,p.y()-2.5,5,5,QPen(Qt::green),QBrush(Qt
         ::green));
   foreach (QPointF p,opC)
      ri<<scene.addEllipse(p.x()-2.5,p.y()-2.5,5,5,QPen(Qt::cyan),QBrush(Qt
         ::cyan));
   foreach (QPointF p,opS)
      ri<<scene.addEllipse(p.x()-2.5,p.y()-2.5,5,5,QPen(Qt::red),QBrush(Qt::
         red));
   scp=-1;

   connect(&scene,SIGNAL(mouseMove(QGraphicsSceneMouseEvent*)),this,SLOT(
      mouseMove(QGraphicsSceneMouseEvent*)));
   connect(&scene,SIGNAL(mousePress(QGraphicsSceneMouseEvent*)),this,SLOT(
      mousePress(QGraphicsSceneMouseEvent*)));
}

MainWindow::~MainWindow()
```

```
{
   delete ui;
}

void MainWindow::mouseMove(QGraphicsSceneMouseEvent *mouseEvent)
{
   if((mouseEvent->buttons()&Qt::LeftButton)&&scp>=0){
      QPointF pos=mouseEvent->scenePos();
      cp[scp]=pos;
      rc[scp]->setRect(pos.x()-2.5,pos.y()-2.5,5,5);
      QPainterPath curve(cp[0]);
      curve.cubicTo(cp[1],cp[2],cp[3]);
      ci->setPath(curve);
      foreach(QGraphicsEllipseItem *gi,ri){
         scene.removeItem(gi);
         delete gi;
      }
      ri.clear();
      QList<QPointF> opI,opC,opS;
      vBezierSingular(cp,opI,opC,opS);
      foreach (QPointF p,opI)
         ri<<scene.addEllipse(p.x()-2.5,p.y()-2.5,5,5,QPen(Qt::green),QBrush
            (Qt::green));
      foreach (QPointF p,opC)
         ri<<scene.addEllipse(p.x()-2.5,p.y()-2.5,5,5,QPen(Qt::cyan),QBrush(
            Qt::cyan));
      foreach (QPointF p,opS)
         ri<<scene.addEllipse(p.x()-2.5,p.y()-2.5,5,5,QPen(Qt::red),QBrush(
            Qt::red));
   }
}

void MainWindow::mousePress(QGraphicsSceneMouseEvent *mouseEvent)
{
   if(mouseEvent->buttons() & Qt::LeftButton){
      QPointF pos = mouseEvent->scenePos();
      scp=-1;
      for(int i=0;i<4;i++){
         if(QLineF(pos,rc[i]->rect().center()).length()<=5)
            scp=i;
      }
   }
}
```

ソースコード 4: mainwindow.ui

```
<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
 <class>MainWindow</class>
 <widget class="QMainWindow" name="MainWindow">
  <property name="geometry">
   <rect>
```

```
   <x>0</x>
   <y>0</y>
   <width>261</width>
   <height>261</height>
  </rect>
 </property>
 <property name="windowTitle">
  <string>MainWindow</string>
 </property>
 <widget class="QWidget" name="centralwidget">
  <widget class="QGraphicsView" name="graphicsView">
   <property name="geometry">
    <rect>
     <x>10</x>
     <y>10</y>
     <width>241</width>
     <height>241</height>
    </rect>
   </property>
   <property name="minimumSize">
    <size>
     <width>100</width>
     <height>100</height>
    </size>
   </property>
   <property name="mouseTracking">
    <bool>true</bool>
   </property>
   <property name="alignment">
    <set>Qt::AlignmentFlag::AlignLeading|Qt::AlignmentFlag::AlignLeft|Qt::
        AlignmentFlag::AlignTop</set>
   </property>
  </widget>
 </widget>
 <resources/>
 <connections/>
</ui>
```